OpenClaw Robotic Arm Control

OpenClaw Robotic Arm Control

1. Course Content
    - Course Overview
    - Learning Objectives
2. Prerequisites
3. Robotic Arm Related Control Interfaces
4. Case Demonstration
    - 4.1 Start OpenClaw Interaction
    - 4.2 Get Current Joint Angles of Robotic Arm
    - 4.3 Simultaneously Control 6 Joints of Robotic Arm
    - 4.4 Control Single Robotic Arm Joint
    - 4.5 Restore Robotic Arm to Preset Initial Pose
    - 4.7 Adjust Camera Viewing Angle
5. Source Code Analysis
    - `ControlArmJointAngles()` — Simultaneously Control 6 Joints
    - `AdjustCameraView()` — Adjust Camera View
    - `InitArmPose()` — Initialize Robotic Arm Pose
    - `ControlSingleArmJointAngle()` — Control Single Joint Angle

1. Course Content

Course Overview

- OpenClaw robotic arm control is one of the core features of the OpenClaw platform, converting AI natural language instructions into robotic arm control operations through the MCP protocol
- Covers 8 types of robotic arm control interfaces: joint angle acquisition, end-effector pose acquisition, multi-joint simultaneous control, single joint control, camera view adjustment, initial pose recovery

Learning Objectives

1. Understand the interface functions of OpenClaw for controlling robotic arms
2. Master the practical methods and use cases for each robotic arm control interface
3. Be able to optimize grasping and placing accuracy by adjusting configuration file parameters

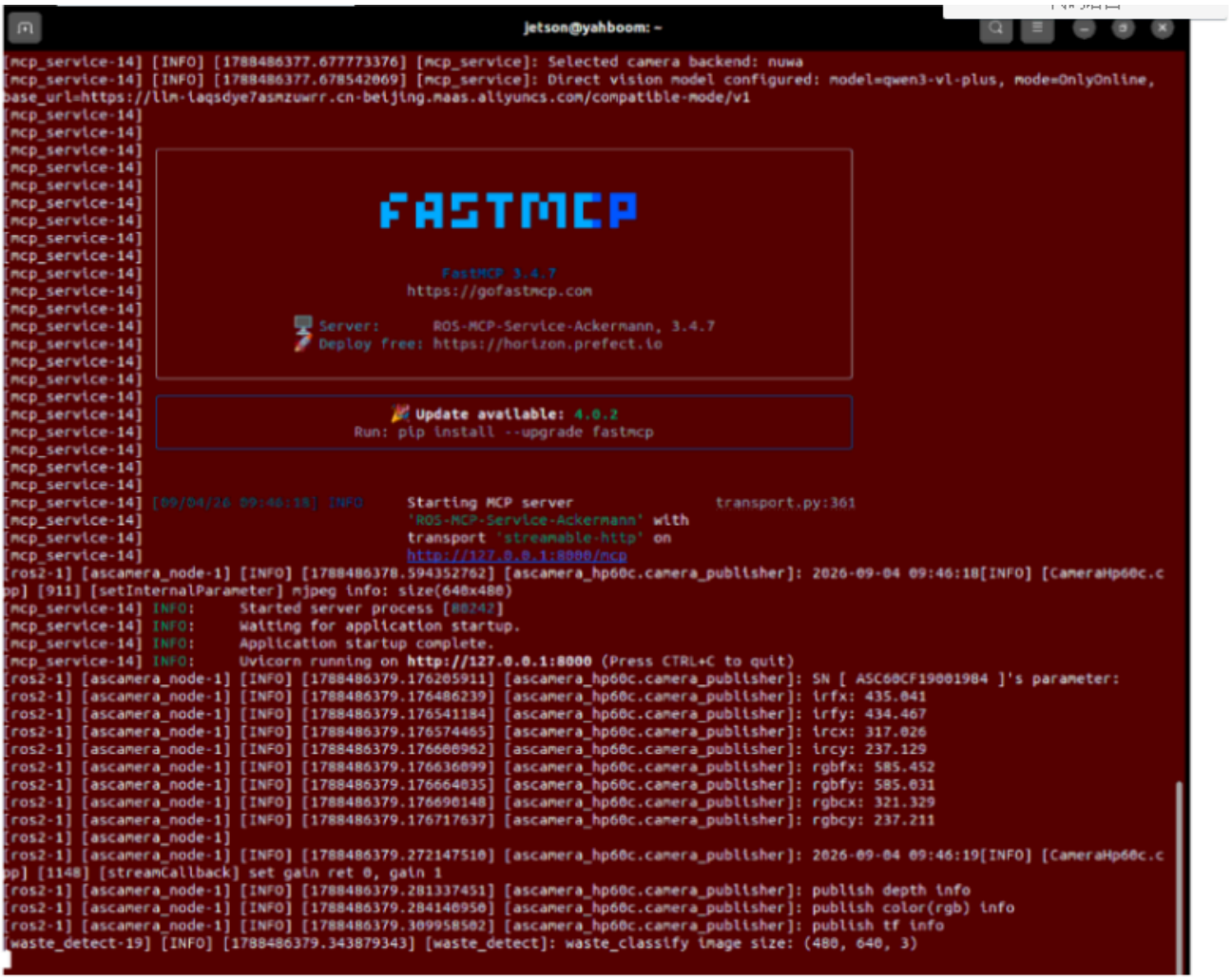
2. Prerequisites

- Start core nodes such as odometry, TF transforms, robotic arm assistance, camera nodes, and MCP service:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py
```

- If you need voice broadcast responses, use the command with the `launch_openclaw_bridge` parameter, otherwise use the basic command above:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py   launch_openclaw_bridge:=true
```



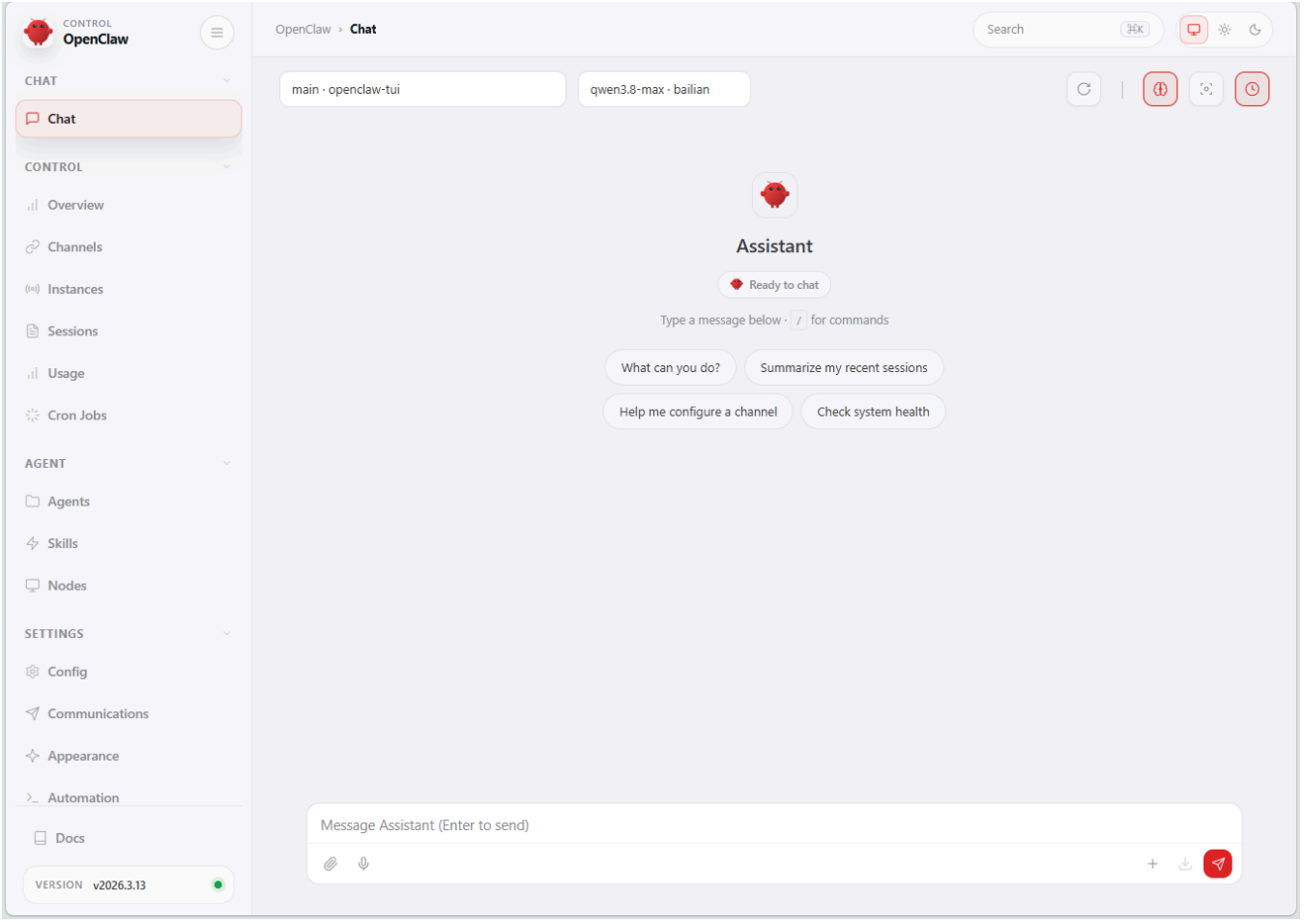
3. Robotic Arm Related Control Interfaces

No.	Tool Name	Function Description
1	GetCurrentArmJointAngles	Get current joint angles of robotic arm
2	ControlSixArmJoint	Simultaneously control 6 joints of robotic arm
3	ControlSingleArmJoint	Control a single robotic arm joint to specified angle
4	InitArmPose	Restore robotic arm to preset initial pose

4. Case Demonstration

4.1 Start OpenClaw Interaction

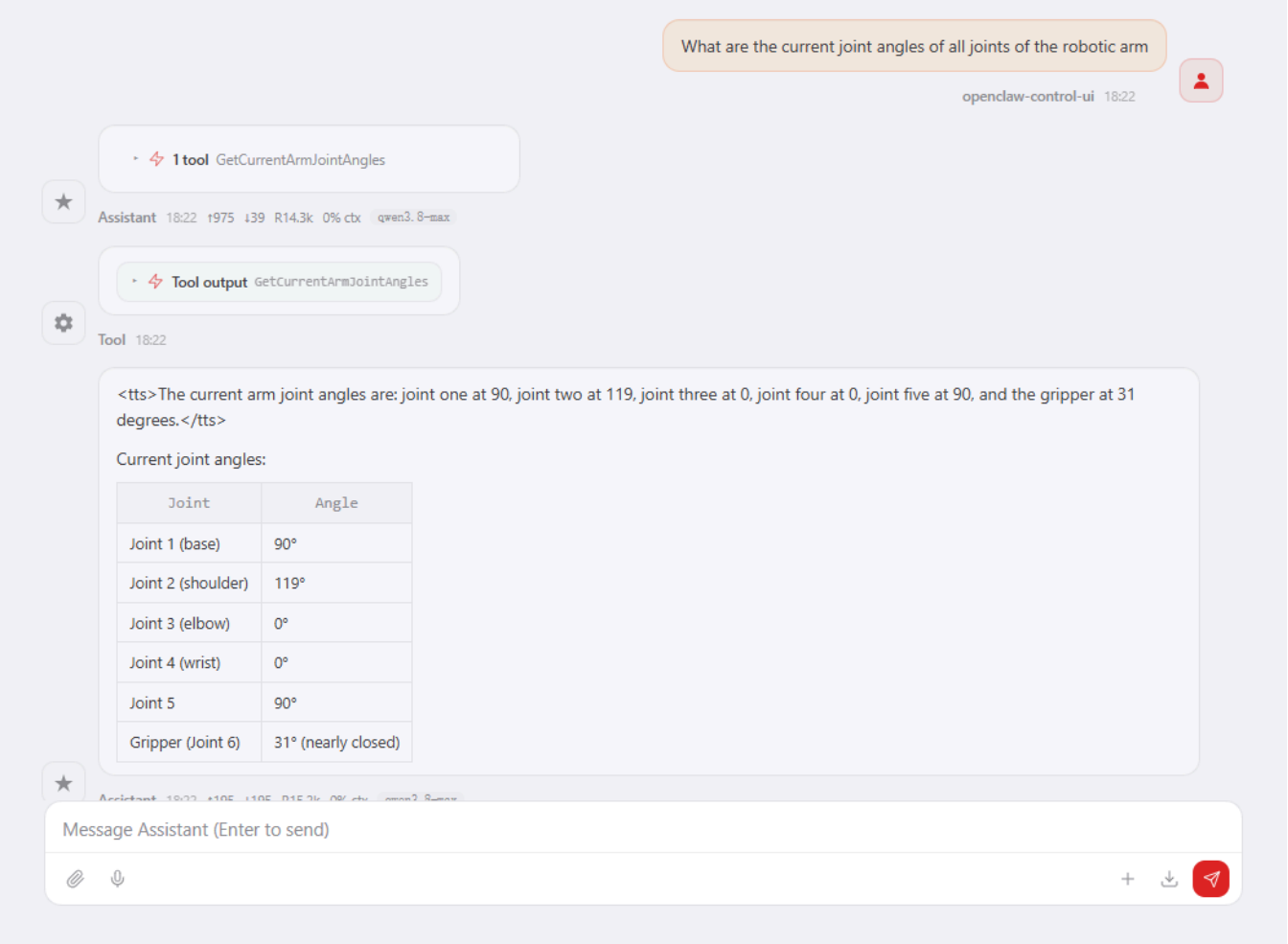
- Use any interaction method to converse with OpenClaw. Here demonstrates using Web Chat



> **Tip**
>
> - Commands can be written according to your own needs. The following are case demonstrations

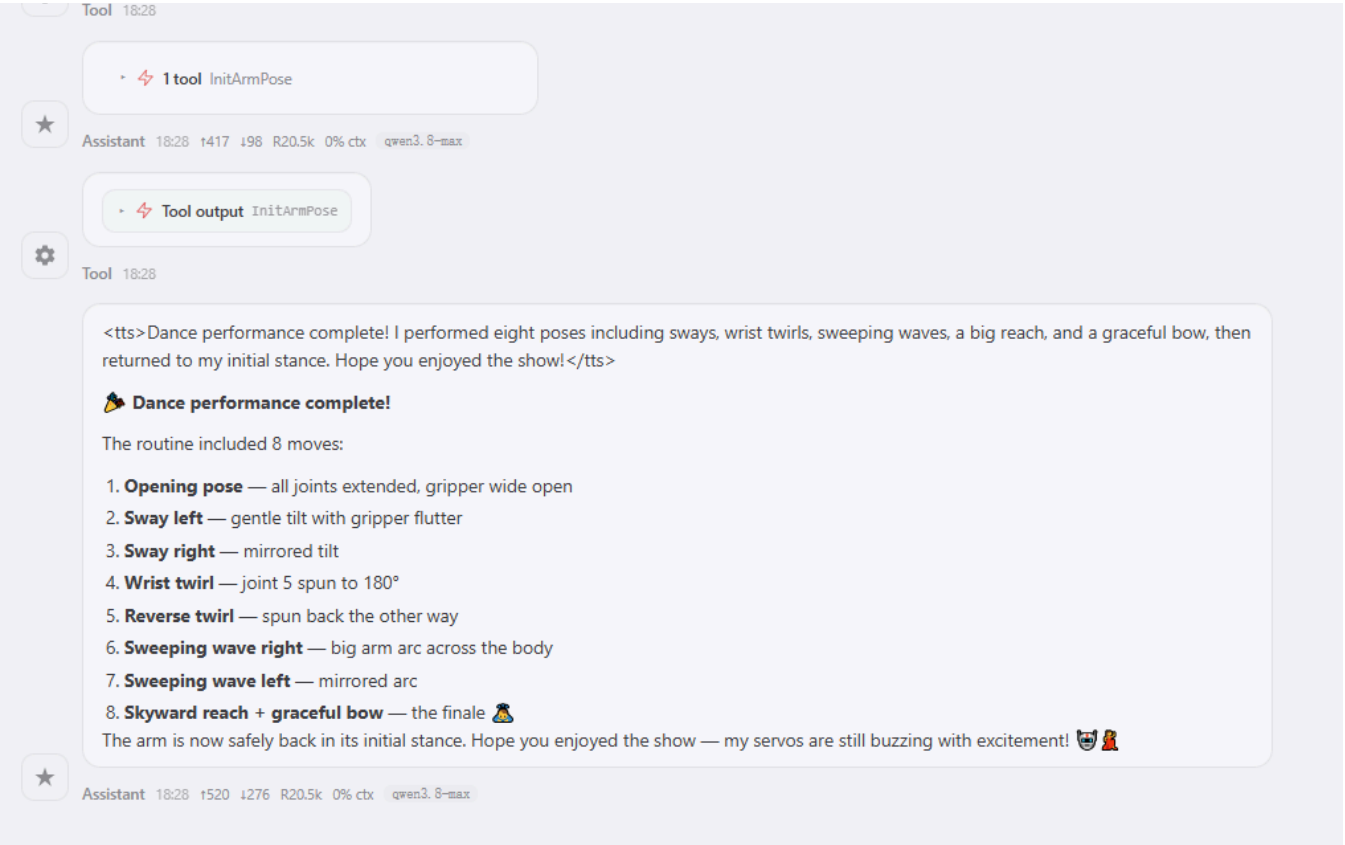
4.2 Get Current Joint Angles of Robotic Arm

- What are the current joint angles of all joints of the robotic arm



4.3 Simultaneously Control 6 Joints of Robotic Arm

- Use the robotic arm to perform a dance for me
- OpenClaw autonomously decides and choreographs robotic arm dance movements. Actual results prevail; different models produce different outputs



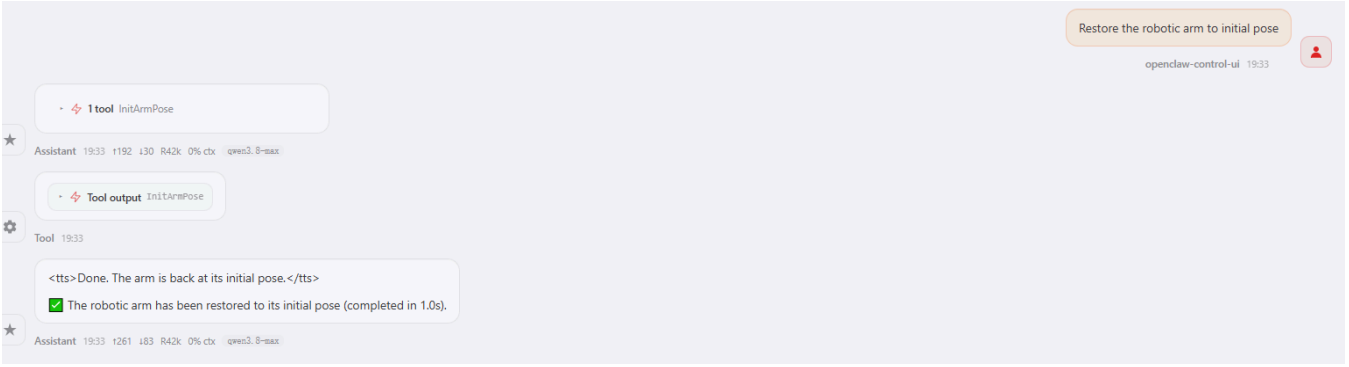
4.4 Control Single Robotic Arm Joint

- Adjust the first joint of the robotic arm to 0 degrees



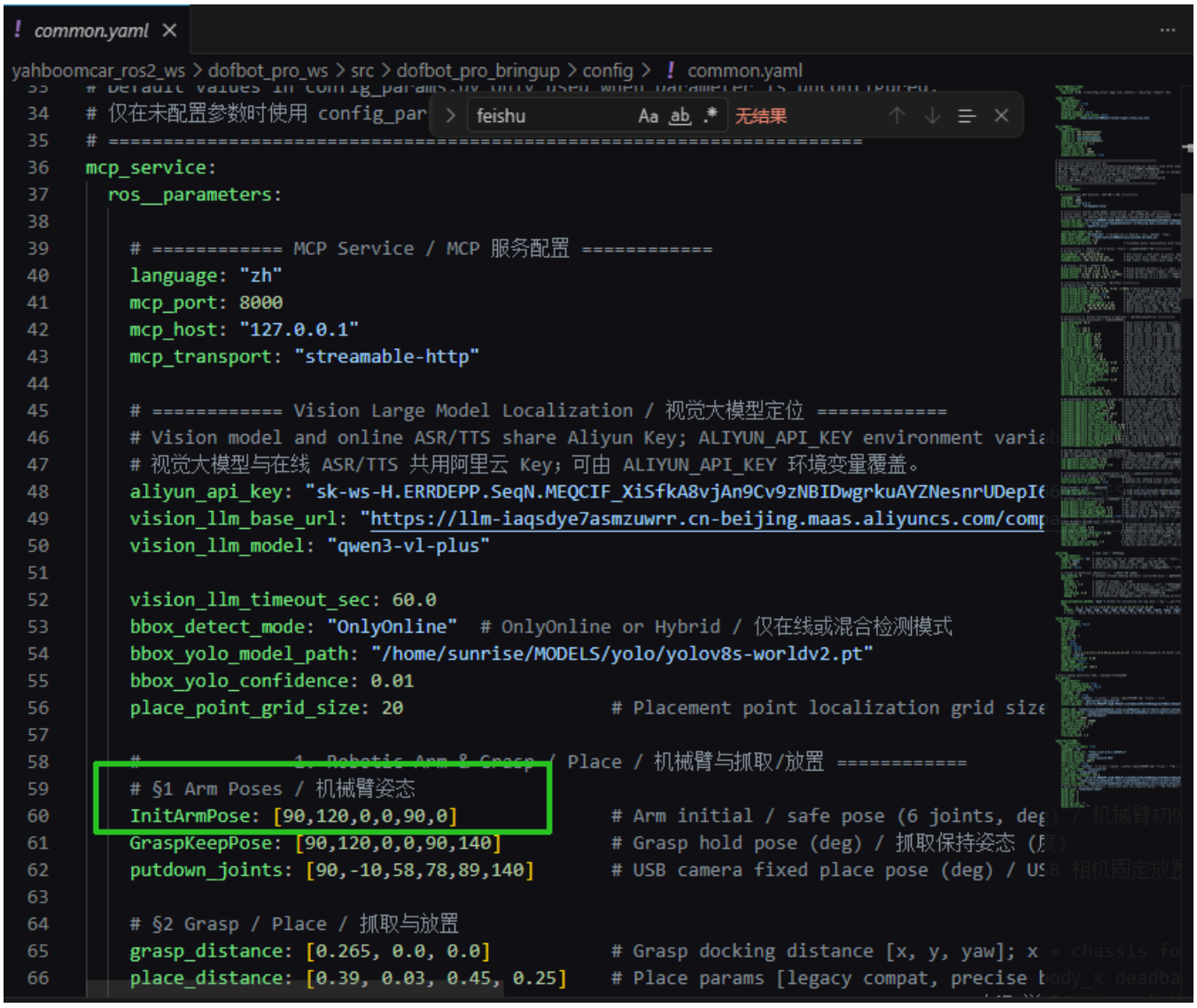
4.5 Restore Robotic Arm to Preset Initial Pose

- Restore the robotic arm to initial pose



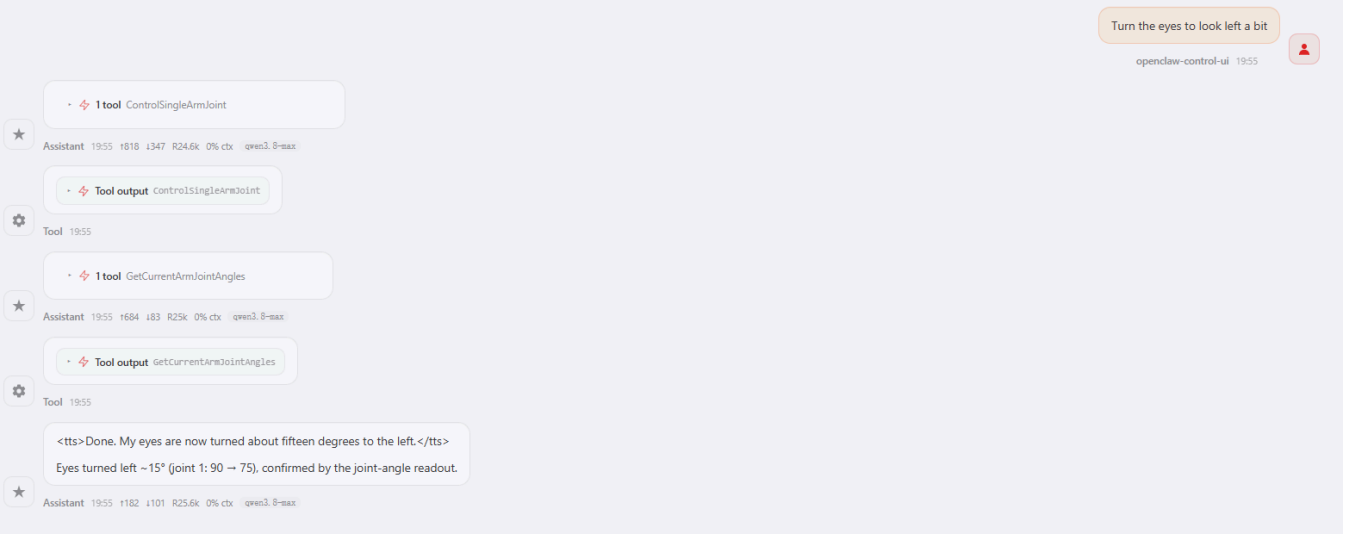
- The initial pose can be modified through the configuration file

```
~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_bringup/config/common.yaml
```



4.7 Adjust Camera Viewing Angle

- Turn the eyes to look left a bit
- This function is used to conveniently adjust the viewing angle when the robotic arm is working to find targets



5. Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/dofbot_pro_ws/src/ackermann_mcp/ackermann_mcp/action_controller.py
```

`ControlArmJointAngles()` — Simultaneously Control 6 Joints

Code Description:

- Construct `ArmJoints` ROS2 message to set target angles for all 6 joints at once, published to the `arm6_joints` topic
- Joint 6 (gripper) undergoes `180 - angle` conversion because the gripper's mechanical zero point and angle definition direction are opposite
- `runtime` parameter controls movement time (milliseconds); larger values result in slower and smoother motion

```
def ControlArmJointAngles(self, arm_joint_1:int, arm_joint_2:int, arm_joint_3:int,
                          arm_joint_4:int, arm_joint_5:int, arm_joint_6:int,
                          runtime:int=2000) -> bool:
    arm_joint = ArmJoints()
    arm_joint.joint1 = arm_joint_1
    arm_joint.joint2 = arm_joint_2
    arm_joint.joint3 = arm_joint_3
    arm_joint.joint4 = arm_joint_4
    arm_joint.joint5 = arm_joint_5
    arm_joint.joint6 = 180 - arm_joint_6  # Joint 6 special handling
    arm_joint.time = runtime               # Movement time (ms)
    self.SixJoint_pub.publish(arm_joint)
    return True
```

`AdjustCameraView()` — Adjust Camera View

Code Description:

- **yaw**: Adjusts via joint 1 to rotate camera left and right; positive value rotates right, negative value rotates left
- **pitch**: Primarily adjusts via joint 4 for up/down tilting; when joint 4 reaches its lower limit (-5°), it automatically switches to joint 2 for pitch compensation
- Automatically calls `SeeWhat()` to refresh the image after adjustment, returns new image path

```
def AdjustCameraView(self, pitch:int=0, yaw:int=0) -> list[bool, str]:
    """Quickly adjust the camera view"""
    self._get_joints()  # Get current joint 1, 2, 4 angles

    if yaw != 0:  # Horizontal adjustment (joint 1 — left/right rotation)
        self.joint_1 += yaw
        self.SingleJoint_pub.publish(ArmJoint(id=1, joint=self.joint_1, time=200))

    if pitch != 0:  # Pitch adjustment (joint 4 — up/down tilt)
        temp_joint = self.joint_4 + pitch
        if temp_joint <= -5:  # Joint 4 reaches lower limit, use joint 2 for pitch
compensation
            self.joint_2 += pitch
            self.SingleJoint_pub.publish(ArmJoint(id=2, joint=self.joint_2, time=200))
        else:
            self.SingleJoint_pub.publish(ArmJoint(id=4, joint=temp_joint, time=200))

    self._reset_joints()
    res = self.SeeWhat()  # Automatically refresh image
    return [True, res[1]] if res[0] else [False, res[1]]
```

`InitArmPose()` — Initialize Robotic Arm Pose

Code Description:

- `InitArmPose`: Restores the robotic arm to a preset safe initial pose, avoiding dangerous or interfering positions
- `ArmJoints()`: Creates a 6-joint message object for simultaneous control of all joints
- `self.init_arm_pose[0]~[5]`: Initial pose angle array read from configuration file `common.yaml`, assigned to joint1~joint6 in order
- `arm_joint.time = 2000`: Sets movement time to 2000ms (2 seconds), ensuring smooth transition to initial pose
- `self.SixJoint_pub.publish(arm_joint)`: Publishes joint angles to `arm6_joints` topic to drive the robotic arm
- `time.sleep(1.0)`: Blocks and waits for 1 second to ensure the robotic arm basically reaches the target position before returning

```python
def InitArmPose(self) -> bool:
    '''Initialize the robotic arm to a safe pose.'''
    # Step 1: Create joint message object
    arm_joint = ArmJoints()

    # Step 2: Set target angles for all 6 joints
    arm_joint.joint1 = self.init_arm_pose[0]
    arm_joint.joint2 = self.init_arm_pose[1]
    arm_joint.joint3 = self.init_arm_pose[2]
    arm_joint.joint4 = self.init_arm_pose[3]
    arm_joint.joint5 = self.init_arm_pose[4]
    arm_joint.joint6 = self.init_arm_pose[5]

    # Step 3: Set movement time (2000ms = 2 seconds)
    arm_joint.time = 2000

    # Step 4: Publish joint angles to topic
    self.SixJoint_pub.publish(arm_joint)

    # Step 5: Wait for robotic arm movement to complete
    time.sleep(1.0)
    return True
```

Function Description:

1. **Safe Pose Initialization**: Restores the robotic arm to a preset safe initial pose, avoiding dangerous or interfering positions
2. **6-Joint Synchronous Control**: Simultaneously controls all 6 joints (joint1~joint6) of the robotic arm to move to target angles
3. **Movement Time Control**: Sets movement duration to 2 seconds via `time=2000`, ensuring smooth transition to initial pose
4. **Blocking Wait**: `time.sleep(1.0)` waits for the robotic arm to basically reach the target position before

returning

`ControlSingleArmJointAngle()` — Control Single Joint Angle

Code Description:

- `ControlSingleArmJointAngle`: Individually controls a specific joint of the robotic arm to a specified angle without affecting other joint states
- `arm_joint_id == 6` branch: Joint 6 is the gripper, performing `angle = 180 - angle` angle flip because the gripper's mechanical zero point and angle definition direction are opposite; after flipping, it conforms to intuitive control (180=fully open, 0=fully closed)
- `ArmJoint(id=arm_joint_id, joint=angle, time=runtime)`: Constructs single joint control message; `id` is joint number (1~6), `joint` is target angle, `time` is movement time (milliseconds)
- `self.SingleJoint_pub.publish(...)`: Publishes single joint control command to topic to drive the corresponding joint

```
def ControlSingleArmJointAngle(self, arm_joint_id: int, angle: int, runtime: int = 1500) ->
bool:
    """ Control a single joint of the robotic arm"""
    # Step 1: Special handling for joint 6 (gripper)
    if arm_joint_id == 6:
        angle = 180 - angle  # Gripper angle flip

    # Step 2: Publish single joint control command
    self.SingleJoint_pub.publish(
        ArmJoint(id=arm_joint_id, joint=angle, time=runtime)
    )
    return True
```

Function Description:

1. **Single Joint Independent Control**: Allows individual control of a specific joint without affecting the states of other joints
2. **Gripper Angle Flip**: Joint 6 (gripper) undergoes special handling with `angle = 180 - angle` because the gripper's 0° and 180° correspond to closed and open states respectively; angle flip is needed to conform to intuitive control
3. **Configurable Movement Time**: `runtime` parameter defaults to 1500ms (1.5 seconds), can be adjusted as needed for movement speed

Parameter Description:

Parameter	Type	Description	Range
`arm_joint_id`	int	Joint ID (1~6)	-
`angle`	int	Target angle (degrees)	Usually 0~180° depending on joint type
`runtime`	int	Movement time (milliseconds)	Default 1500, recommended range 500~3000